# Word Embeddings

## NLP Lecture 2026 — Traitement du Langage Naturel

Christophe Ambroise

2026-03-10

# Words and Vectors

Vector or distributional models of meaning are generally based on a co-occurrence matrix

**Two common co-occurrence matrix**

- the word-document matrix
- the word-word matrix.

# Word-document matrix

In a Word-document matrix, each row represents a word in the vocabulary and each column represents a document from some collection of documents.

Corpus of 5 short sentences about NLP (used as a running example throughout this chapter):

Table 1: Term-document matrix — S1: *"word embeddings represent meaning"* · S2: *"deep neural networks learn representations"* · S3: *"language models process text"* · S4: *"word vectors capture context"* · S5: *"neural language models predict"*

| | S1 | S2 | S3 | S4 | S5 |
|---|---|---|---|---|---|
| **word** | 1 | 0 | 0 | 1 | 0 |
| **embeddings** | 1 | 0 | 0 | 0 | 0 |
| **vectors** | 0 | 0 | 0 | 1 | 0 |
| **neural** | 0 | 1 | 0 | 0 | 1 |
| **language** | 0 | 0 | 1 | 0 | 1 |
| **models** | 0 | 0 | 1 | 0 | 1 |
| **[illegible]** | [illegible] | [illegible] | [illegible] | [illegible] | [illegible] |

# Word-Word matrix

In the Word-context matrix the columns are labeled by words rather than documents.

- each cell records the number of times the row (target) word and the column (context) word co-occur in some context in some training corpus.

## Context

- a context could be the document, in which case the cell represents the number of times the two words appear in the same document.
- smaller contexts are common: generally a window around the word

Word-word matrix for the same corpus (window $L = 1$, symmetric counts):

Table 2: Word-word co-occurrence matrix (window $L = 1$)

| | word | embed-dings | vectors | represent | meaning | capture | context |
|---|---|---|---|---|---|---|---|
| **word** | — | 1 | 1 | 0 | 0 | 0 | 0 |

# Cosine for measuring similarity

The cosine similarity metric (or empirical correlation) between two word vectors $v$ and $w$ (lines of a word-document or word-word matrix)

$$\cos(v, w) = \frac{v^T w}{\|v\|\|w\|}$$

# TF-IDF: Weigthing terms in the vector

## Term frequency (tf)

- Raw frequency $tf_{t,d} = count(t, d)$ is very skewed and not very discriminative.
- Usually squashed by using the $log10$: a word appearing 100 times in a document doesn't make that word 100 times more relevant

$$\text{tf}_{t,d} = \log_{10}(count(t, d) + 1)$$

## Inverse document frequency

The second factor in tf-idf is used to give a higher weight to words that occur only in a few documents.

$$idf_t = \log_{10}\left(\frac{N}{\text{df}_t}\right)$$

where $N$ is the total number of documents, and $df_t$ is the number of documents in which term $t$ occurs.

# TF-IDF: Weigthing terms in the vector

The tf-idf weighted value $w_{t,d}$ for word $t$ in document $d$ thus combines term frequency $tf_{t,d}$ with $idf$:

$$w_{t,d} = tf_{t,d} \times idf_t$$

Applied to our running corpus ($N = 5$ documents):

Table 3: TF-IDF weighted term-document matrix — tf $= \log_{10}(1+1) = 0.30$ for all present terms (each appears once per sentence)

| | S1 | S2 | S3 | S4 | S5 | idf |
|---|---|---|---|---|---|---|
| **word** | 0.12 | 0 | 0 | 0.12 | 0 | 0.40 |
| **embeddings** | **0.21** | 0 | 0 | 0 | 0 | 0.70 |
| **vectors** | 0 | 0 | 0 | **0.21** | 0 | 0.70 |
| **neural** | 0 | 0.12 | 0 | 0 | 0.12 | 0.40 |
| **language** | 0 | 0 | 0.12 | 0 | 0.12 | 0.40 |
| **models** | 0 | 0 | 0.12 | 0 | 0.12 | 0.40 |

# Pointwise Mutual Information (PMI)

An alternative weighting function to tf-idf, PPMI (positive pointwise mutual information), is used for **term(target)-term(context)-matrices**.

- Target word, word we focus on.
- Context words surrounding the target word

The pointwise mutual information between a target word $w$ and a context word $c$ (Church and Hanks 1989, Church and Hanks 1990) is defined as:

$$PMI(w,c) = \log_2 \frac{P(w,c)}{P(w)P(c)}$$

# Positive PMI (called PPMI)

Negative PMI values (which imply things are co-occurring less often than we would expect by chance) tend to be unreliable unless our corpora are enormous.

## Positive PMI

replaces all negative PMI values with zero (Church and Hanks 1989, Dagan et al. 1993, Niwa and Nitta 1994)

$$PPMI(w,c) = \max(\log_2 \frac{P(w,c)}{P(w)P(c)}, 0)$$

Applied to our word-word matrix (16 context pairs total, window $L = 1$):

$$\text{PMI}(\text{embeddings}, \text{word}) = \log_2 \frac{P(\text{emb}, \text{word})}{P(\text{emb}) \cdot P(\text{word})} = \log_2 \frac{1/16}{(2/16)(2/16)} = \log_2 4 = 2.0$$

# Applications of the tf-idf or PPMI vector models

## Principle

- Computing two documents (or word) similarity after transformation.
- Given two documents $d_1$ and $d_2$, the similarity is $\cos(d_1, d_2)$.

## Applications

- **Documents** : information retrieval, plagiarism detection, news recommender systems, and even for digital humanities tasks like comparing different versions of a text to see which are similar to each other.
- **Words** : finding word paraphrases, tracking changes in word meaning, or automatically discovering meanings of words in different corpora.

# Latent Semantic Analysis (LSA)

## Goal and assumptions

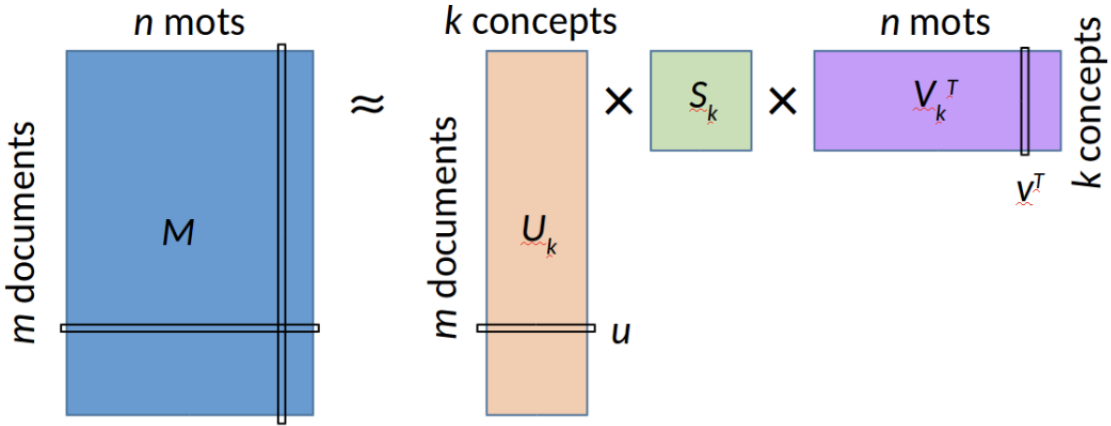
- extracting and representing the underlying meaning of words in a corpus of texts.
- words that occur in similar contexts have similar meanings.

## Principle

LSA uses singular value decomposition (SVD) to identify latent, or hidden, patterns in word co-occurrence data.

# Latent Semantic Analysis

LSA uses SVD on the matrix of word-document matrix to identify the latent concepts (contexts, topics, ...) that underlie the relationships between words in the corpus: $W = USV^T$ with classical low rank approximation $U_k S_k V_k^T$



# Latent Semantic Analysis

- The values on the main diagonal of $S_k$ indicate the 'importance' of each of the k main 'latent concepts' (or factors).
- For each of the document, the corresponding row of $U_k$ allows us to see which concepts are present and with what weights.
- For each of the concept, the associated column of $V_k$ indicates which terms form the concept (and with what weights).

Calculating the similarity between a term and a document involves choosing:

- the row $\mathbf{u}$ corresponding to the document in the matrix $\mathbf{U}_k$,
- the row $\mathbf{v}$ corresponding to the term in the matrix $\mathbf{V}_k$,
- and then computing the product $\mathbf{u}\mathbf{S}_k\mathbf{v}^T$.

To determine the similarity of a term to each of the documents

$$\mathbf{U}_k\mathbf{S}_k\mathbf{v}^T$$

Conversely, we can determine the similarities between a document $\mathbf{u}$ and all the terms through

# Latent Dirichlet Allocation (LDA)

## History

In the context of population genetics, LDA was proposed by J. K. Pritchard, M. Stephens and P. Donnelly in 2000.
LDA was applied in machine learning by David Blei, Andrew Ng and Michael I. Jordan in 2003.

## Principle

Latent Dirichlet Allocation (LDA) is a probabilistic generative model that assumes that

- each document is generated by a mixture of latent topics
- each topic is generated by a mixture of words from the vocabulary.

Documents are represented as random mixtures over latent topics, where each topic is characterized by a distribution over all the words.

# LDA Generative process

For a corpus $D$ consisting of $M$ documents

1. Choose topic parameter vector for document $i$: $\theta_i \sim \mathrm{Dir}(\alpha)$, where $i \in \{1, \dots, M\}$ and $\mathrm{Dir}(\alpha)$ is a Dirichlet distribution with a symmetric parameter $\alpha$ which typically is sparse ($\alpha$).
2. Choose the word parameters vector of topic $k$: $\varphi_k \sim \mathrm{Dir}(\beta)$, where $k \in \{1, \dots, K\}$ and $\beta$ typically is sparse
3. For each of the word positions $i, j$, where $i \in \{1, \dots, M\}$, and $j \in \{1, \dots, N_i\}$

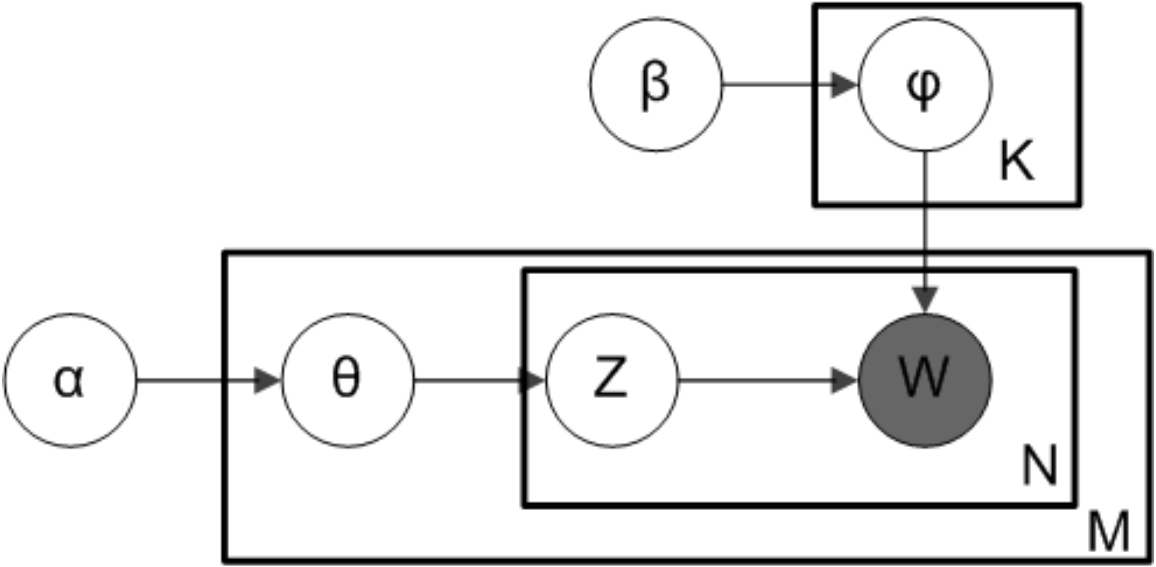
a. Choose a topic $z_{i,j} \sim \mathrm{Cat}(\theta_i)$.
b. Choose a word $w_{i,j} \sim \mathrm{Cat}(\varphi_{z_{i,j}})$.

# Latent Dirichlet Allocation (LDA) in summary

$$\varphi_{k=1\ldots K} \sim \text{Dirichlet}_V(\beta) \tag{1}$$
$$\theta_{d=1\ldots M} \sim \text{Dirichlet}_K(\alpha) \tag{2}$$
$$z_{d=1\ldots M, w=1\ldots N_d} \sim \text{Categorical}_K(\theta_d) \tag{3}$$
$$w_{d=1\ldots M, w=1\ldots N_d} \sim \text{Categorical}_V(\varphi_{z_{dw}}) \tag{4}$$

$$P(W, Z, \theta, \varphi; \alpha, \beta) = \prod_{i=1}^{K} P(\varphi_i; \beta) \prod_{j=1}^{M} P(\theta_j; \alpha) \prod_{t=1}^{N} P(Z_{j,t} \mid \theta_j) P(W_{j,t} \mid \varphi_{Z_{j,t}}),$$

# Latent Dirichlet Allocation (LDA) estimation



# Example of LDA in Action

Corpus of Three Documents

1. "cats dogs pets love"
2. "dogs bark loud outside"
3. "politics government election law"

Step 1: Identify Topics

After running LDA, it may discover two topics (dist. over words):

- Topic 1 (Pets): {cats, dogs, pets, bark, love}
- Topic 2 (Politics): {politics, government, election, law}

Step 2: Assign Topics to Documents

- Document 1 -> $90\%$ Topic 1, $10\%$ Topic 2
- Document 2 -> $80\%$ Topic 1, $20\%$ Topic 2
- Document 3 -> $95\%$ Topic 2, $5\%$ Topic 1

# Pro and Cons

## Advantages of LDA

- Unsupervised Learning: No labeled data needed.
- Interpretable Topics: Extracts meaningful topics from text.

## Limitations of LDA

- Fixed Number of Topics: Must specify the number of topics beforehand.
- Bag-of-Words Assumption: Ignores word order and context.
- Computationally Expensive: Requires approximation methods for inference.

## Applications of LDA

- Topic Modeling: Organizing large text corpora (news articles, research papers).
- Recommendation Systems: Suggesting articles based on topic similarity.
- Text Classification: Assigning labels based on topic distributions.
- Social Media Analysis: Identifying trending topics in posts.

# Word2vec

## Embeddings

- more powerful word representation
- short dense vectors: $d$ ranging from 50-1000, rather than the much larger vocabulary size $|V|$ or number of documents $D$
- dense vectors work better in every NLP task than sparse vectors

## Skip-gram with negative sampling

- The skip-gram algorithm is one of two algorithms in a software package called word2vec
- The other algorithm is CBOW (continuous bag of words)

# Embedding derived from classification

## Data : words and their context (neighboring words ($\pm L$)

... lemon, a [tablespoon of apricot jam, a] pinch ... c1 c2 w c3 c4

## Classification task

Given a tuple $(w, c) \in \mathbb{R}^d \times R^d$ of a target word $w$ paired with a candidate context word $c$ (for example (apricot, jam), or perhaps (apricot, aardvark)) return the probability that $c$ is a real context word (true for jam, false for aardvark):

$$P(y_{wc} = 1|w, c) = 1 - P(y_{wc} = 0|w, c)$$

## Similarity

we rely on the intuition that two vectors are similar if they have a high dot product (after all, cosine is just a normalized dot product). In other words:

# From dot product to logistic regression

## Logit

$$P(y_{wc} = 1|w, c) = \frac{1}{1 + \exp(-c^T w)}$$

## Criterion

The criterion is a kind of likelihood including positive examples (observed association) and negative examples (non observed associations). In fact skip-gram with negative sampling (SGNS) uses more negative examples than positive examples (with the ratio between them set by a parameter $k$).

$$L = \sum_{w,c:y_{wc}=1} \log P(y_{wc} = 1|w, c) + \sum_{w',c':y_{w'c'}=0} \log P(y_{w'c'} = 0|w', c')$$

# Skip-gram training pairs

Sentence: *"word **embeddings** represent meaning"* — target = **embeddings**, window $L = 2$

**Positive pairs** (real context, $y = 1$):

| target | context | label |
|---|---|---|
| embeddings | word | 1 |
| embeddings | represent | 1 |
| embeddings | meaning | 1 |

**Negative samples** ($k = 2$ per positive, sampled $\propto P(w)^{3/4}$):

| target | noise word | label |
|---|---|---|
| embeddings | neural | 0 |
| embeddings | deep | 0 |
| embeddings | process | 0 |

# Two-matrix architecture

Skip-gram maintains two $|V| \times d$ embedding matrices:

$$W = \begin{bmatrix} \leftarrow w_{\text{word}} \rightarrow \\ \leftarrow w_{\text{embeddings}} \rightarrow \\ \vdots \end{bmatrix} \qquad C = \begin{bmatrix} \leftarrow c_{\text{word}} \rightarrow \\ \leftarrow c_{\text{embeddings}} \rightarrow \\ \vdots \end{bmatrix}$$

- **Target matrix** $W$: used when the word is the focus of a training pair.
- **Context matrix** $C$: used when the word appears as context.

The logit for pair $(w, c)$ is a single dot product: $c^T w$.

**Note**

The window size $L$ controls the nature of the learned similarity: $L \leq 2$ captures syntactic relations · $L \geq 5$ captures topical/semantic relations.

# Final embeddings

- skip-gram outputs the target matrix $W$ and the context matrix $C$.
- It's common to just add them together, representing word i with the vector $w_i + c_i$.
- Alternatively we can throw away the C matrix and just represent each word i by the vector $w_i$.
- As with the simple count-based methods like tf-idf, the context window size L affects the performance of skip-gram embeddings, and experiments often tune the parameter L on a devset.

# GloVe: from local windows to global statistics

**Word2Vec** learns from local context windows — each gradient step sees one (target, context) pair. The global co-occurrence structure of the corpus emerges only implicitly after many steps.

**GloVe** (Pennington et al., 2014) makes global statistics *explicit*:

1. Build the full co-occurrence matrix $\mathbf{X}$ where $X_{ij}$ = number of times word $j$ appears in the context of word $i$ across the entire corpus.
2. Directly fit word vectors to this matrix.

> **Note**
>
> On our running corpus, $X_{\text{embeddings,word}} = 1$ and $X_{\text{vectors,word}} = 1$ — GloVe sees this symmetry globally and will assign similar vectors to both words.

# GloVe objective

Learn $w_i, \tilde{w}_j \in \mathbb{R}^d$ and scalar biases $b_i, \tilde{b}_j$ to minimise:

$$J = \sum_{i,j=1}^{|V|} f(X_{ij}) \left( w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2$$

**Weighting function** — downweights rare pairs, caps frequent ones:

$$f(x) = \begin{cases} (x/x_{\max})^\alpha & x < x_{\max} \\ 1 & x \geq x_{\max} \end{cases} \qquad x_{\max} = 100,\ \alpha = 0.75$$

**Intuition**: the dot product of two word vectors should predict their log co-occurrence count: $w_i^T \tilde{w}_j \approx \log X_{ij}$.

# GloVe: meaning from co-occurrence ratios

The key insight is that **ratios** of co-occurrence probabilities encode semantic relations:

| | $P(k \mid \text{ice})$ | $P(k \mid \text{steam})$ | ratio |
|---|---|---|---|
| $k = \text{solid}$ | large | small | $\gg 1$ |
| $k = \text{gas}$ | small | large | $\ll 1$ |
| $k = \text{water}$ | large | large | $\approx 1$ |
| $k = \text{fashion}$ | small | small | $\approx 1$ |

GloVe vectors encode these ratios directly:

$$w_i^T \tilde{w}_k - w_j^T \tilde{w}_k \approx \log \frac{P(k \mid i)}{P(k \mid j)}$$

This explains why linear analogy arithmetic ($w_{\text{king}} - w_{\text{man}} + w_{\text{woman}} \approx w_{\text{queen}}$) works: the vector difference encodes a log probability ratio.

# Word2Vec vs GloVe

| | Word2Vec (SGNS) | GloVe |
|---|---|---|
| Training signal | Local windows, one pair at a time | Global co-occurrence matrix $\mathbf{X}$ |
| Objective | Logistic classification | Weighted least squares |
| Scalability | Streams corpus (SGD) | Requires building $\mathbf{X}$ first |
| Small corpora | Better (no need to store $\mathbf{X}$) | Worse |
| Interpretability | Implicit | Explicit ($\log X_{ij}$ target) |
| Analogy accuracy | High | High (comparable) |

Pre-trained vectors are available for both: `word2vec-google-news-300` (100B tokens), `glove-wiki-gigaword-{50,100,200,300}d` (6B tokens).

**! Important**

**Shared limitation**: both methods produce **one static vector per word** regardless of

# Analogy Tasks

## The Core Idea

Mikolov et al. (2013) observed that semantic relationships are encoded as **linear translations** in the embedding space:

$$\vec{\text{king}} - \vec{\text{man}} + \vec{\text{woman}} \approx \vec{\text{queen}}$$

The vector **difference** captures a relation, and that relation is approximately **consistent** across all pairs sharing the same relation.

## Solving an Analogy

Given the query *"man is to king as woman is to ?"*, compute:

$$\hat{v} = \vec{\text{king}} - \vec{\text{man}} + \vec{\text{woman}}$$

Then find the word $w^*$ whose embedding is closest to $\hat{v}$: